

Python: the Language

Python Introduction

**Datatypes and
Variables**

Lists and loops

Selection

Functions

Organise code & IO

Python Introduction

- The philosophy of Python
 - Programming is not only easy to write but also easy to read.
- Python is *simple* and *small*.
- Python is one of the most popular programming languages that has survived the test of time.

- Python is one of the three languages that students must master
 - Java (C#/C++) : Server/High-performance
 - Python: AI/LLM and SE
 - JavaScript: App development

| Static and Dynamic Language

Program creation has two steps:

1. **Compilation** – translate source code into low-level code.
2. **Execution** – run the translated low-level code.

Compilation

- In this step, a static programming language such as Java or C# binds types to variables and checks types at compile time.
- Dynamic languages like Python bind types to objects at runtime; variables are simply names (labels) referencing objects.

```
// Java compiler (javac) catches type mismatches at compile time
String hello = "world";
// hello = 123; // Compile-time error: incompatible types
char ch = hello.charAt(0); // Valid Java string access (not hello[0])
```

- Python does not check variable types at compile time.
 - A variable can reference any type of object at different times.
 - Type errors surface at runtime when an invalid operation is attempted.

```
hello = "hello"  
hello = 40  
# hello[0] # Runtime error: TypeError: 'int' object is not subscriptable
```

Modern Python type hints

- Type hints document the expected values and let tools such as `mypy` or `pyright` detect many errors before runtime.
- Python 3.12 introduced the `type` statement for clear, generic type aliases.

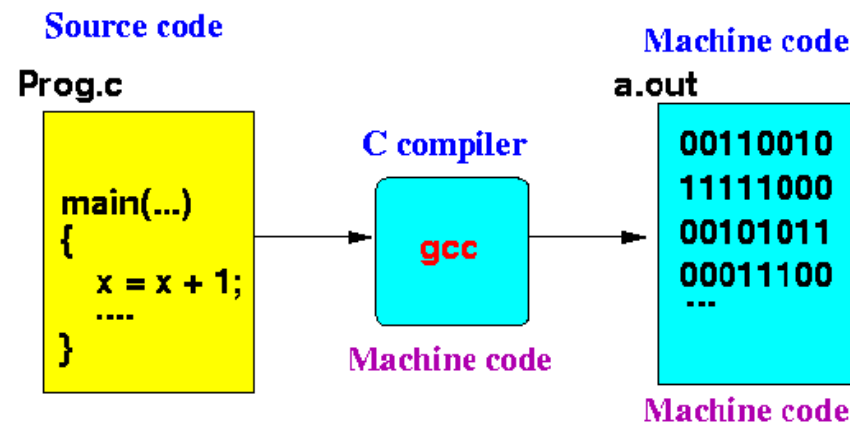
```
type UserId = int
type Result[T] = T | None

def find_user(user_id: UserId) -> Result[str]:
    return "Alice" if user_id == 1 else None
```

Byte code conversion

- In the compilation step, modern programming languages translate high-level code into low-level code.
 - Humans understand high-level code.
 - Machine understands low-level code.

- System programming languages, such as C/C++/Rust, compile high-level code directly to machine code.
- CPU can execute the machine code directly.



- Java or Python translates high-level code into bytecode.
- CPU cannot execute bytecode directly, so we need a VM (Virtual Machine).

Java:	Bytecode:
1 public class Example{	1 public class Example{
2	2
3 int x;	3 i x
4	4
5 static int y;	5 static i y
6	6
7 public int getX() {	7 public getX()i
8 return x;	8 aload 0
9 }	9 getfield Example.x : i
10	10 ireturn
11	11
12 public void setX(int x) {	12 public setX(iIv
13 this.x = x;	13 aload 0
14 }	14 iload 1
15	15 putfield Example.x : i
16	16 return
17	17
18 public int increaseY() {	18 public increaseY()i
19 return y++;	19 getstatic Example.y : i
20 }	20 dup
21	21 iconst_1
22	22 iadd
23	23 putstatic Example.y : i
24	24 ireturn
25	25
26 public void EventHandler() {	26 public EventHandler()v
27 setX(1);	27 aload 0
28 }	28 iconst_1
29	29 invokevirtual Example.setX()v
30	30
31 }	31 }

Execution

- Java's JVM uses a Just-In-Time (JIT) compiler to compile hot bytecode into native machine code during execution.
- CPython compiles source code into bytecode (`.pyc`), and the CPython Virtual Machine executes it.
- Execution speed can be significantly slower than compiled languages for CPU-bound tasks, depending on the workload and implementation.

Python: Pros and Cons

Pros of Python

- Clean, expressive syntax: easy to learn and read.
- Rapid prototyping with no manual compilation step.
- Rich ecosystem of libraries and frameworks (AI, data science, web).
- Strong community and tooling.

Cons of Python

- No mandatory static type checking by default; type errors surface at runtime.
- Optional static type hints (`mypy`, `pyright`) are now widely used to catch bugs early:

```
age: int = 20
name: str = "Alice"
is_active: bool = True

def greet(name: str) -> str:
    return f"Hello, {name}"
```

Slow execution time (No JIT)

- Performance depends on the workload; measure the actual program instead of assuming a fixed speedup.
- PyPy can be about 1.5-3 times faster for long-running, CPU-bound pure Python loops, but may be slower for short scripts, I/O, or C-extension-heavy code.

- CPython 3.13 introduced an experimental, opt-in JIT; its improvements are currently modest rather than multiple times faster.
- Mojo's claim of up to 35,000x is from a specific benchmark, not a general expectation.

Python knows its issues, and is trying to solve the problems.

Datatypes and Variables

- Python has datatypes and variables.
- Python does not check types for variables.

```
# x can reference any time without checking  
x = 10 # 10 is an int type value  
x = "Hello" # "Hello" is a string type value
```

- A datatype (at a high level) is a set of specific values and their properties.
- The number 15 is an integer datatype, and the integer datatype has properties such as '+'.

- String datatype also has a  property, but the action differs (this is called Polymorphism).

```
x = 10; y = 20;  
z = x + y # z = 30
```

```
a = "Hello"; b = "World";  
c = a + b # c = "HelloWorld"
```

- We can use `type()` function to check the type name

```
type("Amsterdam") # -> <class 'str'>
type(1) # -> <class 'int'>
type([1, 2, 3]) # -> <class 'list'>
type({'a': 10}) # -> <class 'dict'>
type({1, 2}) # -> <class 'set'>
type(True) # -> <class 'bool'>
```

Expression

A Python expression computes a value.

- Arithmetic operators include `+`, `-`, `*`, `/`, `//` (floor division), `%` (modulo), `**` (power).

```
print(3 + 4)
print(3 - 4)
print(3 * 4)
print(3 / 4) # returns 0.75 (float type value)
print(3 // 4) # returns 0 (int type value)
print(10 + 20*4) # 90 as multiplication first
print((10+20)*4) # 120
```

Variables

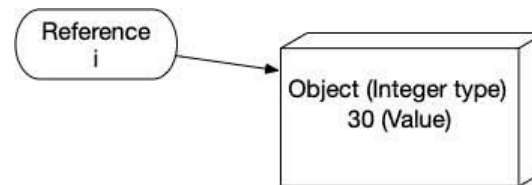
Python is multi-paradigm; everything in Python is an object at runtime.

- No `new` keyword needed.
- Variables bind dynamically to objects.

```
// Java: explicit type declaration and allocation  
Integer i = Integer.valueOf(30);
```

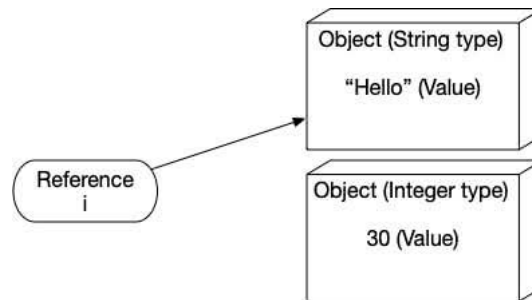
```
# Python: binds name 'i' to int object 30  
i = 30
```

- Variables in Python are references (names bound to objects).
- Assignment binds a name to an object in memory.



```
i = int(30) # 30 is an int object  
i = 30      # same literal representation
```

- In this example, **i** points to an integer object, and can be rebound to reference any object freely.

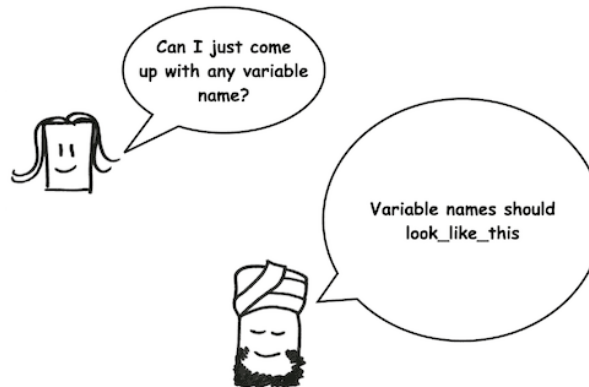


```
i = 30
```

```
i = "Hello" # i now references a str object
```

Naming rules

- Class names conventionally follow PascalCase (PEP 8): `Airplane`, `UserAccount`.
- Variable and function names use snake_case: `user_name`, `total_price`.
- Identifiers cannot start with a digit and cannot contain spaces or special symbols (except `_`).



Floating point number

- Real numbers have infinite mathematical precision, but computers approximate floating-point values using finite bits (IEEE 754).
- Small rounding differences are normal:

$$\frac{2}{3} = 0,66666666...7$$


```
>>> 1.2 + 1.0
2.2
>>> 1.2 - 1.0
0.1999999999999999996 # surprise!
```

| String format and interpolation

Python provides an f-string (format string).

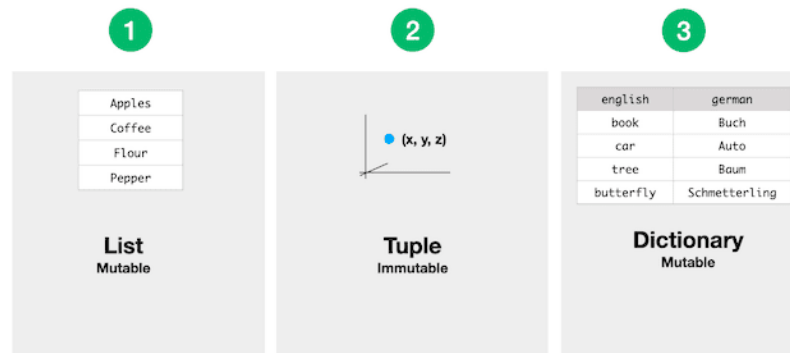
- It is a way of string interpolation (use an expression in a string).

```
name = "world"  
print(f"Hello {name}") # prints out "Hello world"  
print("Hello " + "world") # alternative solution
```

Collections and Loops

Python provides versatile built-in collections:

- **list** (mutable ordered sequence).
- **tuple** (immutable ordered sequence).
- **dict** (mutable key-value mapping).



A List

- We can use the `[ ... ]` literal syntax or `list()` to make a list.
- We use the `append` method to add an item to a list.

```
names = ["Vera", "Chuck", "Dave"]  
names = list() # creates empty list  
names.append("Vera")
```

- We use an index to access a list.
- Python supports for-in-loop.

```
names = ["Vera", "Chuck", "Dave"]  
print(names[0]) # "Vera"  
for name in names:  
    print(name)
```

- Python uses `len()` to get an array's size, not `names.length()`.
- This relies on duck typing: `len()` calls the list's hidden `__len__` method.

```
names = ["Vera", "Chuck", "Dave"]  
names.__len__() # get the length
```

```
len(names) # ducktyping
```

- Python has built-in functions, such as `len()` and `print()`.
- There are many built-in functions.

		Built-in Functions		
<code>abs()</code>	<code>delattr()</code>	<code>hash()</code>	<code>memoryview()</code>	<code>set()</code>
<code>all()</code>	<code>dict()</code>	<code>help()</code>	<code>min()</code>	<code>setattr()</code>
<code>any()</code>	<code>dir()</code>	<code>hex()</code>	<code>next()</code>	<code>slice()</code>
<code>ascii()</code>	<code>divmod()</code>	<code>id()</code>	<code>object()</code>	<code>sorted()</code>
<code>bin()</code>	<code>enumerate()</code>	<code>input()</code>	<code>oct()</code>	<code>staticmethod()</code>
<code>bool()</code>	<code>eval()</code>	<code>int()</code>	<code>open()</code>	<code>str()</code>
<code>breakpoint()</code>	<code>exec()</code>	<code>isinstance()</code>	<code>ord()</code>	<code>sum()</code>
<code>bytearray()</code>	<code>filter()</code>	<code>issubclass()</code>	<code>pow()</code>	<code>super()</code>
<code>bytes()</code>	<code>float()</code>	<code>iter()</code>	<code>print()</code>	<code>tuple()</code>
<code>callable()</code>	<code>format()</code>	<code>len()</code>	<code>property()</code>	<code>type()</code>
<code>chr()</code>	<code>frozenset()</code>	<code>list()</code>	<code>range()</code>	<code>vars()</code>
<code>classmethod()</code>	<code>getattr()</code>	<code>locals()</code>	<code>repr()</code>	<code>zip()</code>

Tuples

- Tuples are immutable sequences.
- Created with `(...)`, accessed with the `[]` indexing operator.

```
x = (1, 2, 'a')  
# Mutating raises AttributeError: 'tuple' object has no attribute 'append'  
# x.append(4)  
print(x[0]) # 1
```

- Tuples have a `__len__` method, so the built-in `len()` function works on them thanks to duck typing.

```
x = (1, 2, 'a')  
print(len(x)) # 3  
x.__len__() # 3
```

Dictionary

- A Python dictionary is a collection (mapping) of key-value pairs.
- Created with `{key: value, ...}`.

```
x = {'a': 10, 'b': 20}
print(x['a'])
x.update({'c': 30})
for k in x: print(k) # print keys
for k in x: print(x[k]) # print values
```

- Both lists and dictionaries use `[]` for access, but their structures differ:
 - A `list` is a contiguous dynamic array indexed by zero-based integers.
 - A `dict` is a hash table mapping arbitrary hashable keys to values.

```
num_list = [10, 20, 30]
print(num_list[0]) # 10

num_dict = {'a': 10, 'b': 20}
print(num_dict['a']) # 10
```

- The `in` operator checks if a key is in the dictionary.

```
x = {'a': 10}
if 'b' in x:
    x['b'] += 20
else:
    x['b'] = 20 # creates a new key
```

Selection

- Relational Expression
- If statement

Relational Expression

- Python has relational operators: `>`, `<`, `==`, `!=`, `>=`, `<=`.
- Logical operators `'and'`, `'or'`, `'not'` combine expressions.

```
5 > 4 # True  
5 > 4 and 5 > 6 # False
```

- Python uses `==` to compare strings (checks value equality).

```
"Jim" == "Sam" # Like Java's .equals()
```

Selection: If and Match

- Python uses `if`, `elif`, and `else` for conditional branching.
- Python 3.10+ also introduced `match / case` for structural pattern matching.

```
x = 3
if x == 3:
    print("x is 3")
elif x == 4:
    print("x is 4")
else:
    print("Other")
```

Conditional Expression

- Python supports inline conditional expressions:

```
# Python conditional expression  
value = 10  
x = 3 if value == 10 else 20
```

```
// Java equivalent (ternary operator ? :)  
int value = 10;  
int x = (value == 10) ? 3 : 20;
```

Functions

Python supports functions.

- Define a function with `def`, a name, parameters in parentheses, and a body.
- Use `return` to send back a value from the function.

- We can call the function by giving arguments that match the parameters.

```
def add(x, y):  
    return (x + y)  
print(add(1,2))
```

```
def add(x = 1, y = 2): # default argument  
    return (x + y)  
print(add()) # same
```

Nested Functions

- Python supports nested functions (functions in a function).

```
def complex_add(x,y):  
    def add(x, y): return (x + y)  
    return add(x,y) + add(x,y)  
complex_add(1,2) # add(1,2) + add(1,2) -> 1+2+1+2  
complex_add(y=2, x=1)
```

Closure

- Nested functions can access variables from their outer function—this is called a closure.
- Closures let inner functions use those variables even after the outer function ends.

```
def outer():  
    message = "Hello, world!"  
    def inner():  
        print(message)  
    return inner  
  
greet = outer()  
greet()  # Output: Hello, world!
```

- `inner` accesses `message` from `outer`, forming a closure.
- `greet` retains `message` even after `outer` ends.

| Argument Unpacking (*args, **kwargs)

- Use `*` to unpack an iterable into positional arguments.
- Use `**` to unpack a dict into keyword arguments.

- Argument unpacking makes code cleaner and function calls more flexible.

```
x = [1, 2]
print(add(*x)) # equivalent to add(1, 2)
y = {'x': 10, 'y': 20}
print(add(**y)) # equivalent to add(x=10, y=20)
```

Organize code & IO

- File read/write
- Error Handling

File read/write

- We can use the built-in `open()` function to open a file and read/write strings.

```
text = "Hello, this is a text file!"
file = open("hello.txt", "w") # "w" for write
file.write(text)
file.close()

file = open("hello.txt", "r")
for line in file.readlines(): # read lines as strings
    print(line.strip())
file.close()
```

Error Handling

- **SyntaxError**: Detected during parsing/compilation before bytecode execution (e.g., missing closing parenthesis or colon).
- **Runtime Exceptions**: Occur while the program is actively executing (e.g., `FileNotFoundError`, `ZeroDivisionError`, `TypeError`).

```
# SyntaxError (caught before running):  
>>> print("Hell"  
SyntaxError: '(' was never closed  
  
# Runtime Exception (caught while running):  
>>> file = open("nonexistent.txt", "r")  
FileNotFoundError: [Errno 2] No such file or directory: 'nonexistent.txt'
```

Runtime errors that go unhandled crash the application for end users.

- Use `try/except` to catch and handle runtime exceptions gracefully.
- Use `with` statements (context managers) to ensure resources like open files are closed automatically, even if an exception occurs.

```
# Graceful error handling with try/except
try:
    with open("hello2.txt", "r") as file:
        content = file.read()
except FileNotFoundError as e:
    print(f"File not found: {e}")
```